

Comparaison expérimentale des algorithmes de chiffrement symétrique



Performance et robustesse : architecture x86 (Windows) vs ARM (Raspberry Pi)

Agenda



01

Nature du système & architecture



02

Protocole expérimental



03

Validation fonctionnelle — KAT



04

Résultats de performance



05

Robustesse cryptographique



06

Synthèse & recommandations



Nature du système

- **Moteur expérimental**

pas une application utilisateur

- **Aucune interface graphique**

entrée = paramètres · sortie = CSV

- **Conçu pour isoler la variable matérielle**

même code, deux plateformes, résultats comparables

Répétabilité garantie — PyCryptodome délègue aux mêmes routines C compilées sur les deux OS. Les octets sont des octets.





Protocole expérimental

	Laptop x86	Raspberry Pi
CPU	Intel/AMD x86-64	ARM Cortex-A
OS	Windows 11	Raspberry Pi OS
AES-NI	✓ Oui	✗ Non
RAM	~16 GB	4 GB
Contexte	Multitâche	Minimaliste

5

algorithmes*AES · DES · 3DES · Twofish · ChaCha20*

8

paliers*tailles : 64 B → 16 384 B*

100

répétitions *$IC_{95} = 1.96 \times \sigma / \sqrt{n}$*

n = 100 choisi au seuil où l'IC à 95 % se stabilise — au-delà, le gain de précision est marginal par rapport au coût CPU.

Pourquoi Python ?

Critère	Python	C++	Java
Performance brute	⚠ Moyenne	✓ Élevée	⚠ Moyenne
Biais d'optimisation	✓ Faible	✗ Compilateur	✗ JIT / JVM
Reproductibilité inter-plateformes	✓ Excellente	⚠ Variable	⚠ Variable
Accès primitives bas niveau	✓ Via PyCryptodome	✓ Direct	⚠ Limité
Encodage des données	✓ bytes bruts	✓ Contrôlable	✗ JVM dépendant

Sur C++ — C++ aurait mesuré les optimisations du compilateur autant que les algorithmes. L'objectif est de comparer les algos entre plateformes, pas les compilateurs.

Sur l'encodage — Messages générés en bytes bruts via `os.urandom()` — aucune conversion str/unicode, aucun biais Linux/Windows possible.



Known Answer Tests — KAT

Avant toute mesure de performance : prouver que les implémentations sont correctes.

Algorithme	Mode	Référence NIST/RFC	Statut
AES-128	ECB	NIST FIPS 197	✓ PASS
AES-256	GCM	NIST SP 800-38D	✓ PASS
DES	ECB	NIST SP 800-67	✓ PASS
3DES	ECB / CBC	NIST SP 800-67	✓ PASS
ChaCha20	Stream	RFC 8439	✓ PASS
AES	CBC / CTR	NIST SP 800-38A	✓ PASS

Toutes les implémentations produisent les sorties exactes attendues par les standards — les résultats de performance qui suivent sont fiables.



AES — Advanced Encryption Standard

- Chiffrement par blocs de **128 bits**
- Clés : **128 / 192 / 256 bits** → 10, 12 ou 14 tours
- Chaque tour : **SubBytes → ShiftRows → MixColumns → AddRoundKey**
- Standard NIST depuis **2001** (Rijndael), universellement déployé

AES-NI

Instructions matérielles Intel/AMD depuis 2010

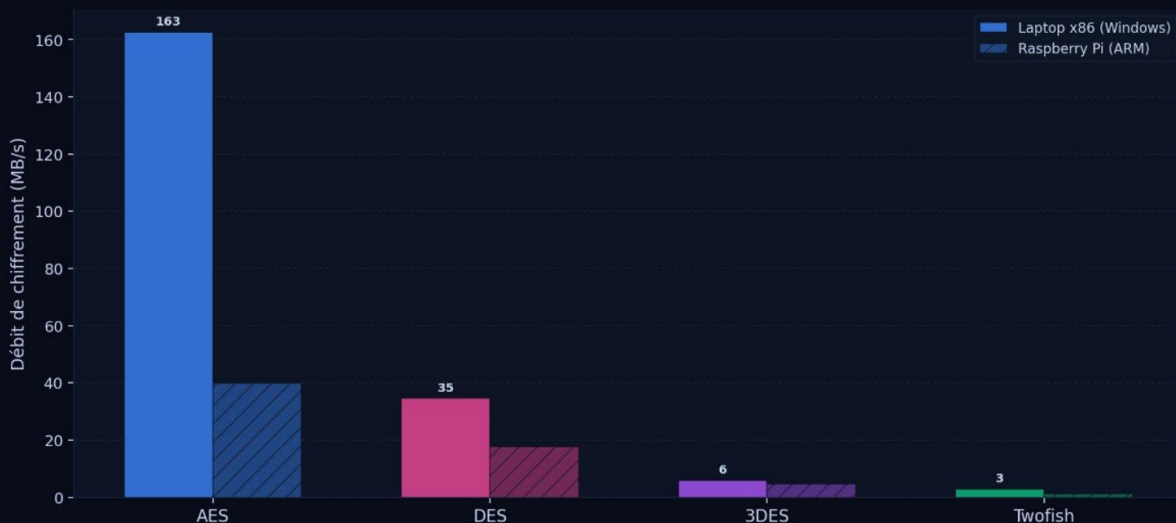
- 1 tour AES complet
= **1 cycle CPU**
- Impact mesuré :
x86 est 4.1× plus rapide que le Pi
- Sans AES-NI :
le Pi calcule en logiciel pur → 4× plus lent

→ L'écart x86 / Pi n'est pas dû à la vitesse du CPU — c'est AES-NI.



AES — Débit global : x86 vs Raspberry Pi

Comparaison 1 — Débit de chiffrement : Laptop x86 vs Raspberry Pi
(mode ECB · 4096 octets · meilleure clé par algorithme)



163 MB/s

AES sur x86

vs 40 MB/s sur Pi — écart dû à AES-NI exclusivement

4.08×

ratio AES x86 / Pi

le plus grand écart de tous les algorithmes testés

3 MB/s

Twofish sur x86

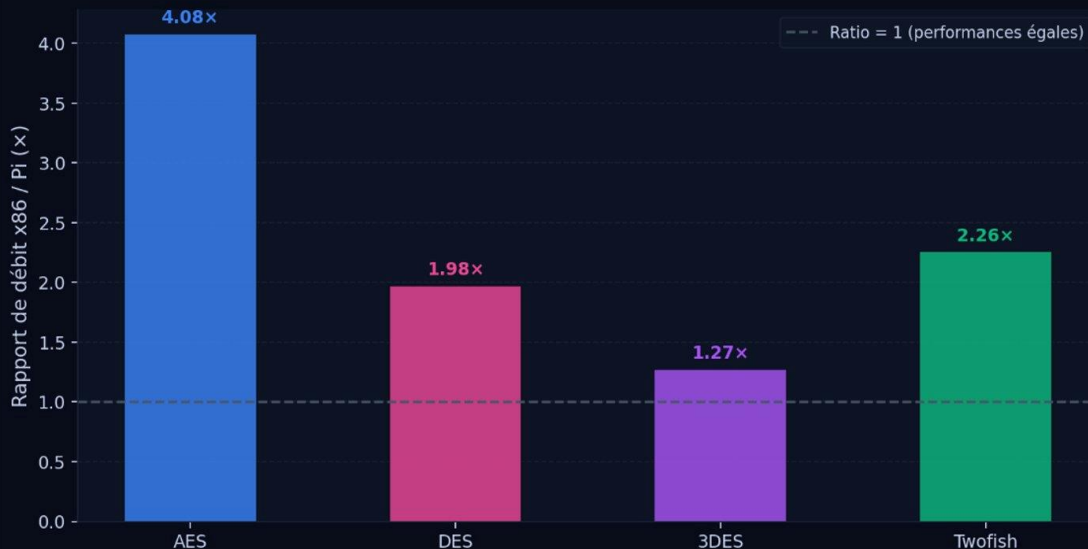
147× plus lent qu'AES — complexité du key schedule

AES sur x86 domine tout — l'écart n'est pas dû à la vitesse du CPU. C'est AES-NI. Sans cette instruction, le Pi ferait le même travail à 4× moins vite.



AES — L'écart x86 / Pi expliqué

Comparaison 2 — Rapport de performance x86 vs Raspberry Pi
(mode ECB · 4 096 octets · valeur > 1 = x86 plus rapide)



4.08x

ratio AES x86 / Pi

le plus grand écart — dû à AES-NI exclusivement

1.27x

ratio 3DES x86 / Pi

lent partout — rien à accélérer matériellement

2.26x

ratio Twofish x86 / Pi

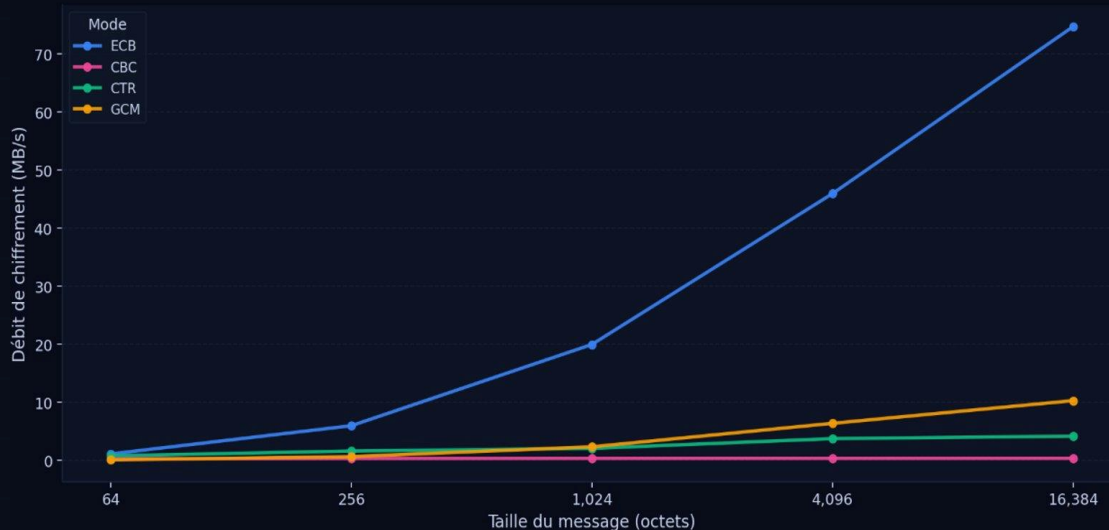
complexité du key schedule pénalise les deux plateformes

AES-NI sur x86 = 1 tour complet en 1 cycle CPU. Sans ça, le Pi fait le même travail en logiciel pur → 4x plus lent. Aucun autre algo n'a ce levier matériel.



AES — Le piège des modes d'opération

Figure 3 — Comparaison des modes d'opération (AES-128)
(plateforme : laptop Windows x86)



ECB 163 MB/s

Rapide mais cryptographiquement cassé

GCM 25 MB/s

Recommandation 2026 — confidentialité + auth

CTR 6.6 MB/s

Parallélisable, bon compromis

CBC 0.73 MB/s

Chiffrement séquentiel — 223× plus lent qu'ECB

ECB = 163 MB/s — le résultat le plus élevé du benchmark. C'est aussi le seul mode cryptographiquement cassé. Optimiser sans comprendre la sécurité est une faute professionnelle.



ChaCha20 — Conçu pour le logiciel pur

- **Chiffrement de flux**

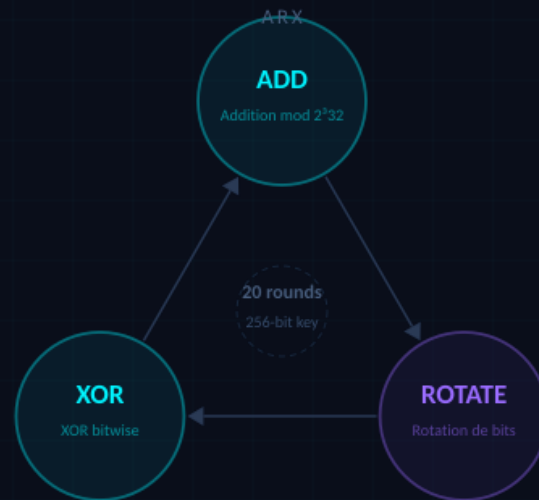
pas de blocs — opère bit par bit sur le keystream

- **Clé unique : 256 bits**

pas de variantes 128/192 — simplicité par design

- **Opérations ARX**

Addition · Rotation · XOR — toutes natives au CPU



Constant-time par design — ADD, ROTATE, XOR s'exécutent en exactement 1 cycle CPU, peu importe les données. Pas de S-Box, pas de timing attack. C'est pourquoi Android et TLS 1.3 mobile choisissent ChaCha20.



ChaCha20 vs AES — Pourquoi ce choix existe

ChaCha20 a été conçu précisément pour corriger les faiblesses d'AES sur ARM.

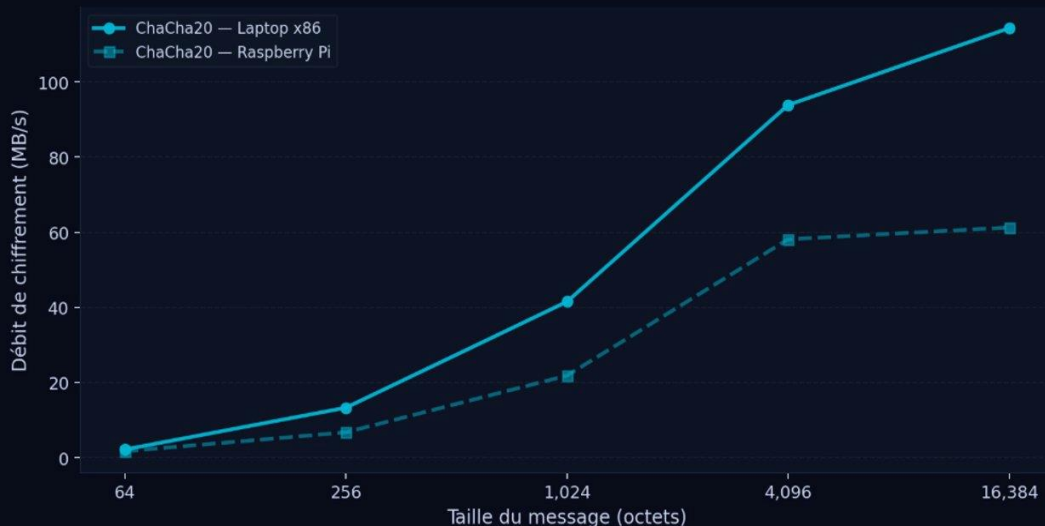
Critère	AES	ChaCha20
Opérations	S-Box (table lookup)	ARX — opérations natives
Accélération HW	AES-NI requis	Aucune nécessaire
Vulnérabilité timing	Possible sans AES-NI	Constant-time par design
Performance ARM	Pénalisé sans NI	Natif et efficace
Débit sur Pi (mesuré)	6.4 MB/s (GCM)	61 MB/s — 9.5× plus rapide

Sur Pi, ChaCha20 (61 MB/s) surpasse AES-GCM (6.4 MB/s) par 9.5× — pour tout déploiement IoT, mobile ou ARM, ChaCha20 est le choix évident. Ce n'est pas une alternative à AES, c'est le standard sur ARM.



ChaCha20 — L'algorithme qui refuse l'inégalité

Comparaison 5 — ChaCha20 : Laptop x86 vs Raspberry Pi
(aucune accélération matérielle sur les deux plateformes — ARX logiciel pur)



114 MB/s

ChaCha20 sur x86

vs 61 MB/s sur Pi — ratio 1.87× seulement

1.87×

ratio x86 / Pi

AES = 4.08× — ChaCha20 est 2× plus équitable

9.5×

avantage sur Pi

ChaCha20 (61 MB/s) vs AES-GCM Pi (6.4 MB/s)

Sur Pi, ChaCha20 surpasse AES-GCM par 9.5× — Bernstein a conçu cet algorithme précisément pour ce scénario : un monde où tous les appareils n'ont pas d'accélération crypto dédiée. C'est le standard sur Android et TLS 1.3 mobile.



DES, 3DES, Twofish — Pourquoi ils ont perdu

DES

MORT

40.7 MB/s

x86 · 18.3 MB/s Pi

Clé effective

56 bits

Cassé en

22h en 1998 (EFF)

Accélération HW

Aucune

Mort depuis 1998 — présent ici pour contexte historique.

3DES

DÉPRÉCIÉ

12.3 MB/s

x86 · 7.5 MB/s Pi

Sécurité effective

112 bits

Coût

3 passes séquentielles

NIST

Déprécié 2017, interdit 2023

Le pire des deux mondes — lent ET faiblement sécurisé.

Twofish

OBSOLÈTE

2.82 MB/s

x86 · 1.29 MB/s Pi

Finaliste AES 1998

Perdu contre Rijndael

Problème

Key schedule trop complexe

vs AES

147× plus lent sur x86

Fascinant académiquement, jamais adopté industriellement.

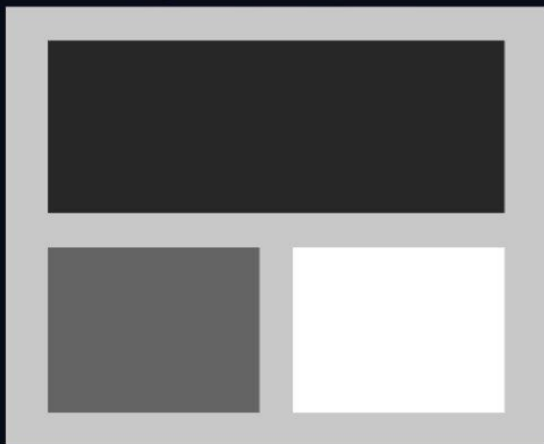
AES : 163 MB/s vs Twofish : 2.82 MB/s — un facteur **57×**. Ces trois algorithmes sont là pour comprendre pourquoi AES a gagné — pas pour être utilisés.



ECB — Le mode le plus rapide est le plus dangereux

Figure 8 — Vulnérabilité du mode ECB
Les patterns de l'original sont préservés dans ECB — masqués dans CBC
(b) Chiffré AES-ECB
⚠ Les patterns restent visibles !

(a) Image originale
(régions uniformes visibles)

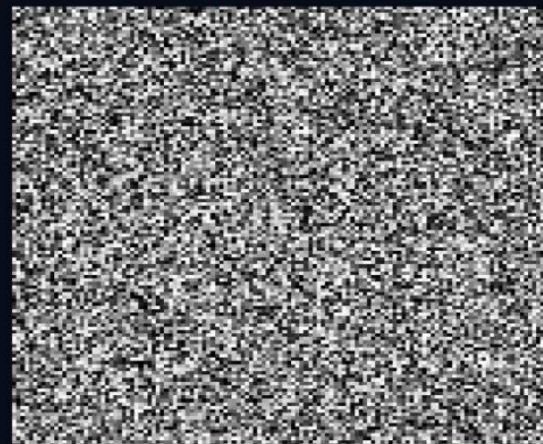


ECB chiffre chaque bloc **indépendamment** avec la même clé



Deux blocs de plaintext identiques → **deux blocs de ciphertext identiques**

(c) Chiffré AES-CBC
✓ Aucun motif structurel visible



Les patterns structurels **survivent au chiffrement** — la clé n'a pas besoin d'être cassée

ECB = 162 MB/s — le résultat le plus élevé du benchmark. C'est aussi le seul mode cryptographiquement cassé. Optimiser sans comprendre la sécurité est une faute professionnelle.



Effet d'avalanche — La santé cryptographique

Comparaison 4 — Effet d'avalanche : x86 vs Raspberry Pi
(propriété mathématique — les scores doivent être indépendants de la plateforme)



Idéal = 0.500 — modifier 1 bit en entrée doit changer exactement 50% des bits en sortie. AES, DES, 3DES et Twofish atteignent tous cet idéal. ChaCha20 dépasse 0.5 pour des raisons de design, pas de faiblesse.

Algorithme	Score	Verdict
AES	0.4997	Parfait
3DES	0.5004	Excellent
Twofish	0.5004	Excellent
DES	0.4982	Acceptable
ChaCha20	0.5948	Anomalie

ChaCha20 à 0.594 n'est pas une faille — c'est une



Sensibilité aux clés — Quand le key schedule défaille

Figure 4b — Comparaison de l'effet d'avalanche : flip texte clair vs flip clé
(tous modes et tailles confondus)



Algorithme	Score clé	Verdict
AES	0.500	Parfait
Twofish	0.500	Excellent
ChaCha20	0.594	Voir sl.13
DES	0.438	Faible
3DES	0.436	Faible

DES à 0.438 — 64 clés "faibles" documentées depuis les années 90 produisent des sorties

AES : 0.500 parfait vs DES : 0.438 — DES n'est pas seulement lent et cryptographiquement faible, son key schedule est structurellement déficient. Ce n'est pas un artefact de mesure.

Stabilité — Le Raspberry Pi plus fiable que le laptop ?

Comparaison 6 — Stabilité des mesures : IC95 · Laptop x86 vs Raspberry Pi
(ECB · 4 096 octets · meilleure clé | valeur faible = mesures stables)



11.28 MB/s

IC95 AES sur x86

vs 2.0 MB/s sur Pi — le laptop est 5.6× plus erratique

≈ 0

IC95 Twofish

Stable sur les deux — trop lent pour que le bruit soit perceptible

6.38 MB/s

IC95 ChaCha20 x86

ARX sans matériel dédié = sensible au scheduler OS

Vitesse brute ≠ fiabilité — Pour un protocole temps-réel ou un HSM, la stabilité compte autant que le débit. Le Pi est plus lent mais plus prévisible. Ce sont deux cas d'usage différents.



Synthèse — Comment choisir en 2026 ?



JAMAIS ECB · DES · 3DES — peu importe le contexte, peu importe la performance. Ce ne sont pas des compromis acceptables en 2026.



Synthèse — Quel algorithme choisir en 2026 ?

Contexte	Algorithme	Mode	Raison
Serveur / cloud	AES-256	GCM	AEAD, NIST, TLS 1.3
IoT / ARM / Mobile	ChaCha20	Poly1305	ARX, constant-time
Données en masse	AES-256	CTR	Parallélisable
À éviter	DES, 3DES	—	Déprécié / cassé
INTERDIT	Tout algo	ECB	Patterns visibles

Conformes à NIST SP 800-175B et RFC 8446 (TLS 1.3)

AES-GCM sur x86, ChaCha20 sur ARM. Jamais ECB, jamais DES ou 3DES dans du nouveau code. Tout ce qui précède dans cette présentation était la preuve empirique de ces quatre lignes.



Conclusion — Ce qui a été livré, ce qui vient ensuite

✓ TN3 — Livré

- 5 algorithmes implémentés et validés (KAT NIST)
- 4 modes d'opération (ECB, CBC, CTR, GCM) + Stream
- Expérimentations sur 2 plateformes : x86 Windows + Pi ARM
- ~600 mesures de débit collectées (n=100 par config.)
- Analyse : avalanche, sensibilité aux clés, stabilité CI95, ECB

→ TN4 — En préparation

- Analyse approfondie des hypothèses de recherche
- Comparaison avec la littérature (Stallings, NIST SP 800-175B)
- Rapport final — interprétation complète des résultats
- Conclusions sur les choix architecturaux
- Recommandations formelles

Le choix d'un algorithme de chiffrement *n'est pas qu'une décision de sécurité — c'est une décision d'architecture, de plateforme, et de contexte d'utilisation.*

Agenda



01 Nature du système & architecture



02 Protocole expérimental



03 Validation fonctionnelle — KAT



04 Résultats de performance



05 Robustesse cryptographique

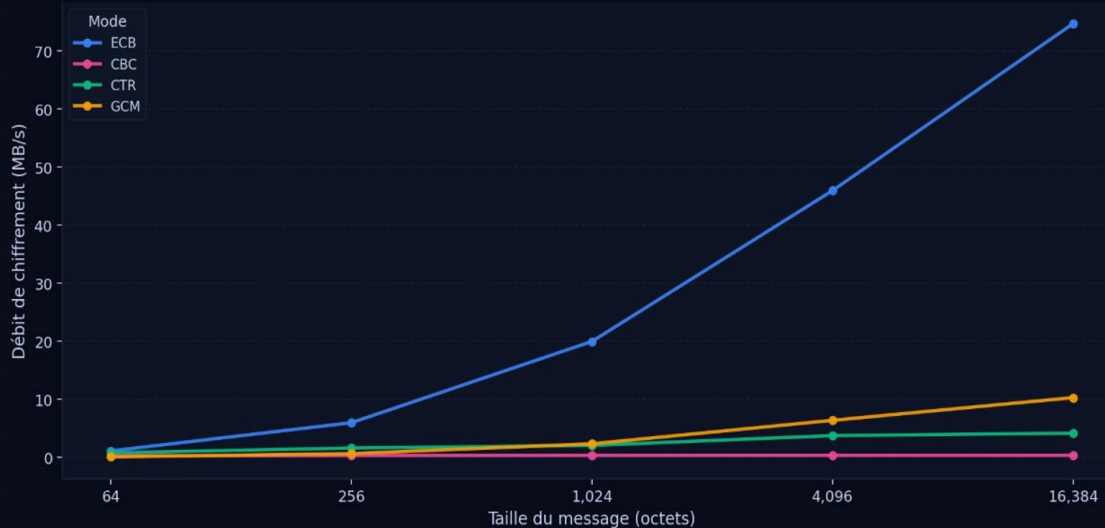


06 Synthèse & recommandations



AES — Le piège des modes d'opération

Figure 3 — Comparaison des modes d'opération (AES-128)
(plateforme : laptop Windows x86)



ECB 163 MB/s
Rapide mais cryptographiquement cassé

GCM 25 MB/s
Recommandation 2026 — confidentialité + auth

CTR 6.6 MB/s
Parallélisable, bon compromis

CBC 0.73 MB/s
Chiffrement séquentiel — 223× plus lent qu'ECB

ECB = 163 MB/s — le résultat le plus élevé du benchmark. C'est aussi le seul mode cryptographiquement cassé. Optimiser sans comprendre la sécurité est une faute professionnelle.